



RESUMEN CLASE ANTERIOR



Operaciones

Todos los lenguajes de programación permiten realizar operaciones con la información.

Operaciones de tipo matemáticas: suma, resta, multiplicación, etc.

Operaciones de tipo lógicas: and, or, not, etc.

Operaciones de comparación: menor que, mayor que, etc.

Operaciones con texto: concatenar, extraer, etc.

Paradigmas de programación

Cuando hablamos de paradigmas de programación nos referimos al enfoque y reglas que utiliza un lenguaje de programación para llevar a cabo las tareas.

Es así entonces que podemos clasificar a los lenguajes de programación mediante el paradigma que implementa.

Tipos de paradigmas de programación

- **Paradigma imperativo:** sucesión de instrucciones concretas.
- Paradigma declarativo: sin algoritmos, se describe el resultado que se espera.
- **Paradigma de programación orientada a objetos:** modelos de objetos que representan elementos del problema a resolver.
- Paradigma reactivo: analiza cambios en el flujo de datos.

Paradigma Imperativo

Subtipos:

- **Programación estructurada:** el flujo de control se define mediante estructuras condicionales, bucles, etc
- **Programación procedimental:** se utilizan funciones o procedimientos que se ejecutan cada vez que se necesiten
- **Programación modular:** divide un programa en módulos o subprogramas



CLASE 06

P00 –JavaScript



Programación Orientada a Objetos

Permite:

- Modelar el mundo real
- Interactuar con el entorno computacional
- Construir componentes de software reutilizables y fácilmente extensibles
- Modificar y ampliar la implementación de estos componentes sin afectar la estructura

Programación Orientada a Objetos

Permite construir fácilmente sistemas complejos a partir de componentes individuales.

Se crean unidades funcionales, extensibles y genéricas.

Estas unidades son representadas mediante clases.

Aspectos básicos

- Estructura modular basada en objetos
- Abstracción de datos
- Gestión automática de memoria
- Clases
- Herencia
- Polimorfismo y enlace dinámico
- Herencia múltiple y repetida

Programación Orientada a Objetos

El método de implementación es mediante colecciones de **objetos**, donde cada uno representa una instancia de alguna clase, y cuyas **clases** son miembros de jerarquías de clases unidas a través de una relación de herencia.

P00 – Clases

Las clases especifican:

- Estructura de datos: son variables que definen la característica de una clase. Al instanciarse un objeto con valores particulares determinan el estado del objeto. Estas variables son llamadas atributos o propiedades.
- Métodos: son funciones creadas por el usuario que definen el comportamiento que tendrán todos los objetos que se instancien

P00 – Objetos

Los objetos son instancias de una clase.

Al momento de instanciarse se debe definir su estado (darle valores particulares a las variables determinados por la estructura de datos de la clase).

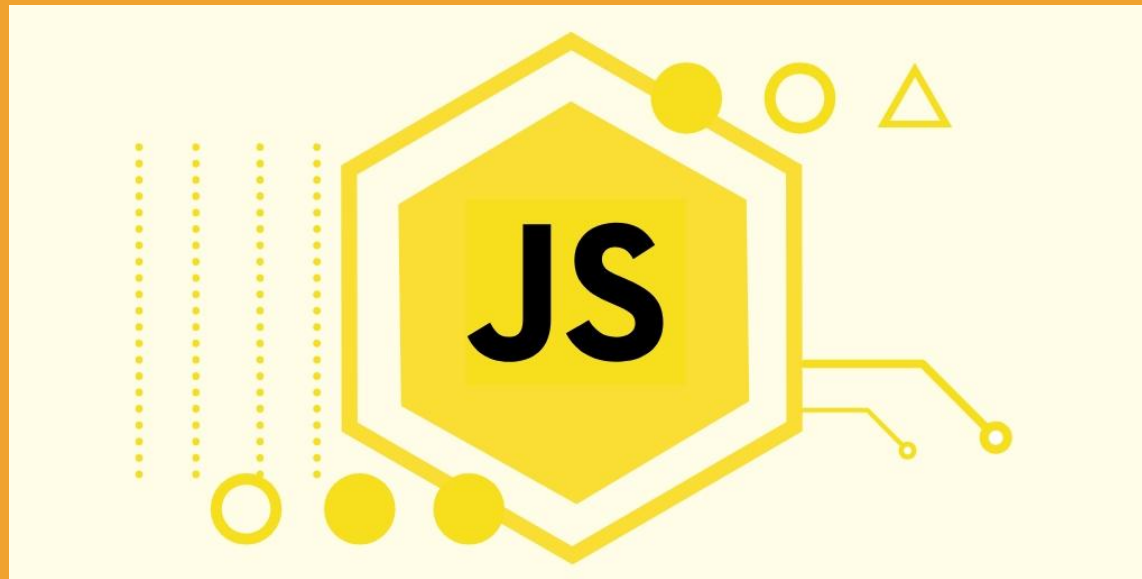
Su comportamiento será en base a los métodos de la clase a la cual pertenece y como cambia su estado con el uso de los mismos.

P00 – Métodos

Representa el comportamiento de los objetos y especifica la forma en que se controlan los datos del mismo (cambio de estado).

Los métodos más comunes que encontramos son: constructor y destructor (siempre presentes aunque no los defina el usuario), de modificación (para cambiar el estado de los objetos y de selección (para ver el estado del objeto).

JAVASCRIPT



JavaScript

Es un lenguaje que usaremos del lado del cliente que permite agregar funcionalidad e interactividad a una página web.

Es un lenguaje interpretado y solo requiere de un navegador para funcionar.

JavaScript

A diferencia de HTML y CSS, en el código de JavaScript se deben respetar las minúsculas y mayúsculas (case sensitive).

Los nombres compuestos tienen la notación camelCase, donde la primera letra de la segunda palabra está en mayúsculas.

Ejemplo: `document.backgroundColor`

JavaScript

Tenemos 3 formas de incorporar código JavaScript:

- En HTML usando las etiquetas `<script>` `</script>`
- Como reacción a un evento
- Incorporando un archivo externo con extensión `.js`

Instrucciones de JavaScript

Todas las instrucciones terminan con un punto y coma ;

```
document.write("texto");
```

Imprime texto en el body

```
window.alert("texto");
```

Muestra un pop-up tipo notificación con el texto indicado

Instrucciones de JavaScript

En JavaScript la sintaxis para acceder a un atributo o método de una clase/objeto se separa con un punto. Ejemplo:

```
document.write("texto");
```

Nombre de la clase

Nombre del método



JavaScript – Ejemplos incorporación

Usando las etiquetas <script>

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scal
  <title>Document</title>
  <script>
    document.write('<h1>Hola mundo</h1>');
  </script>
</head>
<body>
  <script>
    document.write('<p>Lorem ipsum....</p>');
  </script>
</body>
</html>
```

JavaScript – Ejemplos incorporación

Las etiquetas de script pueden estar en el head o en cualquier parte del body.

Para poder mantener la correcta estructura HTML, se recomienda usar dentro del body, dentro de la etiqueta que corresponda.

Hola mundo

Lorem ipsum....

JavaScript – Ejemplos incorporación

Como reacción a un evento. Un evento es algo que le sucede a un elemento tras la interacción del usuario.

Ejemplo:

```
<input type="button" evento="códigoJS">
```

JavaScript – Ejemplos incorporación

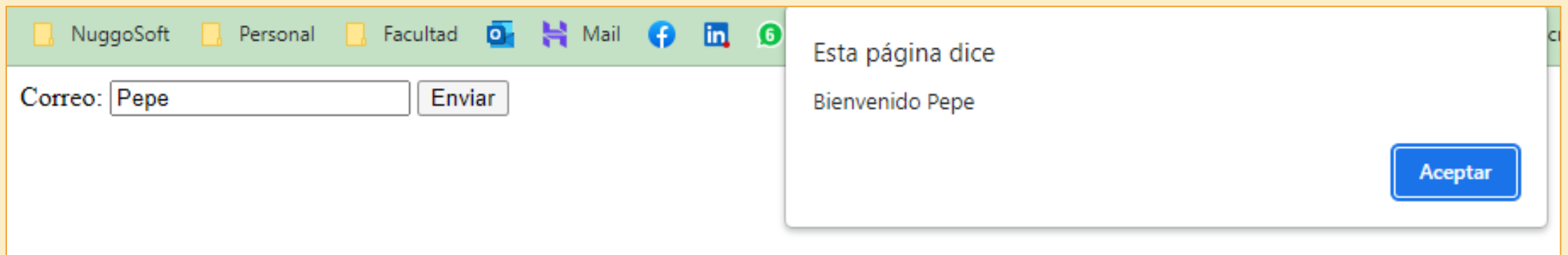
Evento onclick, detecta el click en el elemento button

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <form action="algo.php" method="post" name="formulario">
    <label for="mail">Correo: </label>
    <input type="mail" name="correo" id="mail">

    <input type="button" onclick="window.alert('Hola ' +
    formulario.correo.value)" value="Enviar">
  </form>
</body>
</html>
```

JavaScript – Ejemplos incorporación

Al hacer clic en el botón se ejecuta el código JS que está entre comillas, en este caso la instrucción `window.alert`. Se mostrará el texto que están entre comillas concatenado con el valor del input email (haciendo uso de los `name` de los elementos del formulario). La concatenación (unión de textos) en JS es con el signo más `+`



JavaScript – Ejemplos incorporación

Incorporando un archivo externo .js

```
<body>  
  <script src="script.js" type="text/javascript"></script>  
</body>
```

Y creando un archivo con extensión .js

```
JS script.js  
document.write("<h2>Hola mundo 2</h2>");
```

Ejercicio

Realice un formulario con un campo para el nombre y otro campo para el apellido.

Agregue un botón que diga Saludar. Programe el mismo con JavaScript para que al presionar el botón se muestre una ventana de alerta con el mensaje:

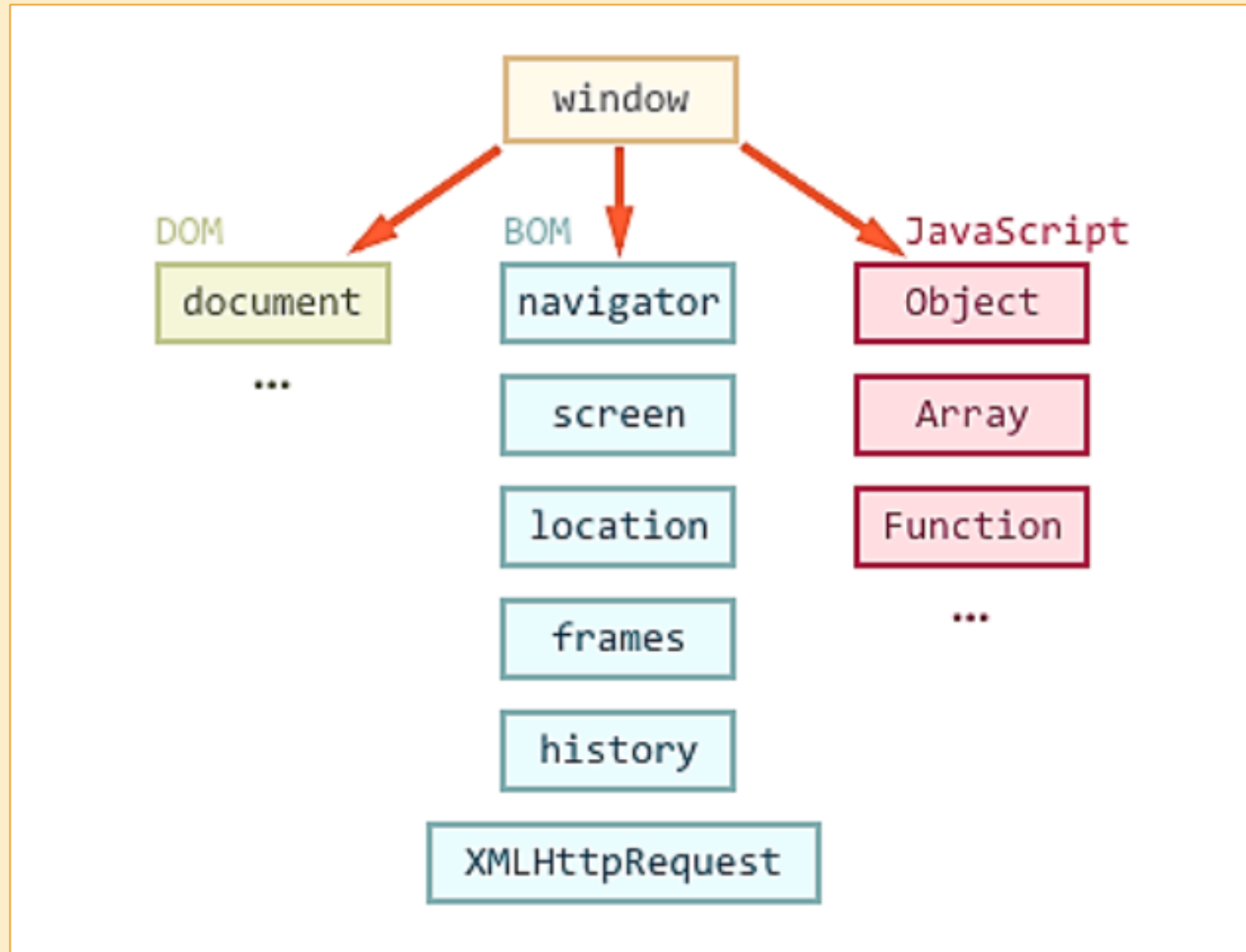
Bienvenido Apellido, Nombre

JavaScript – DOM y BOM

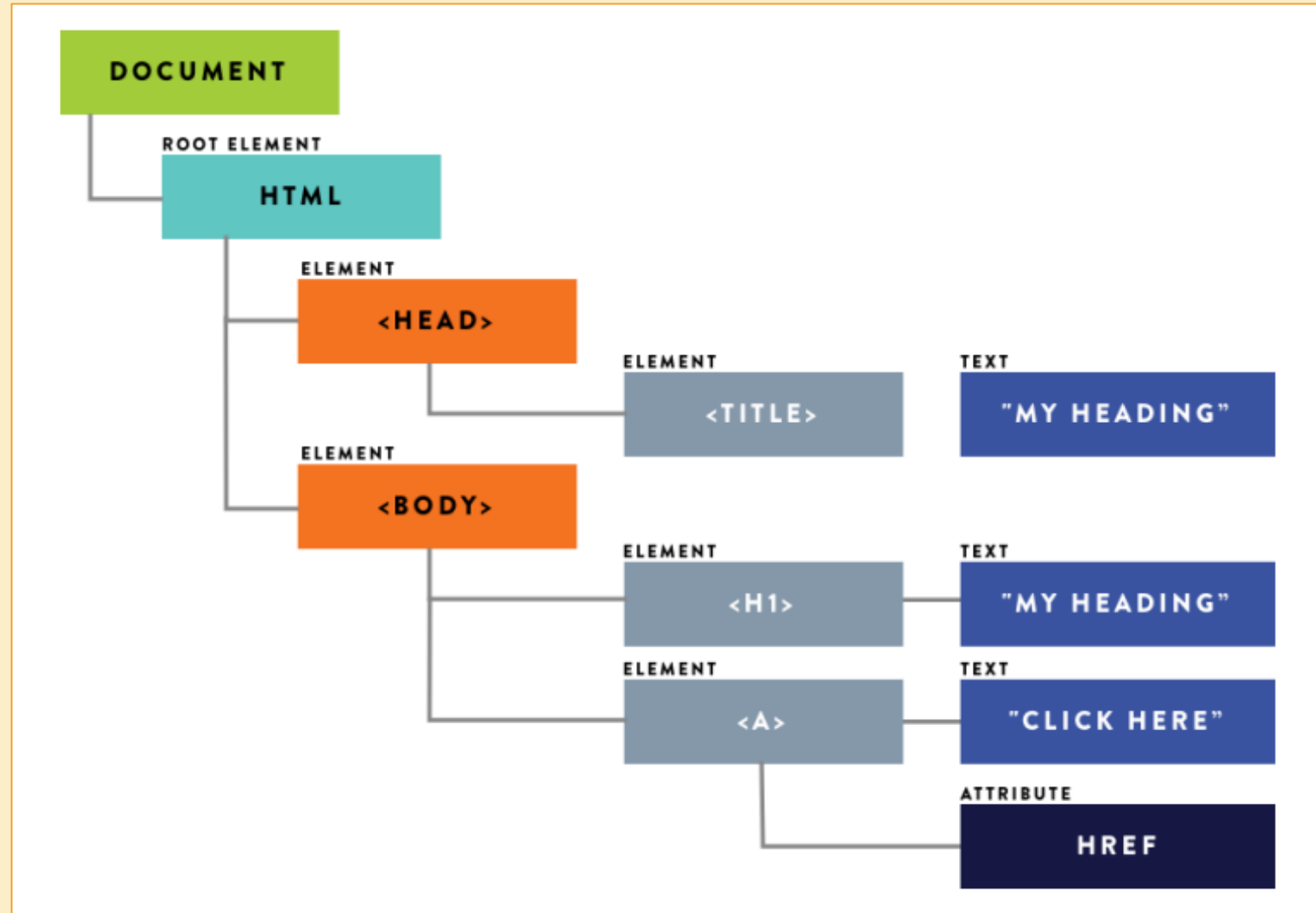
Para aprender a trabajar con las funciones de JS debemos entender que es el DOM y BOM.

- DOM (Document Object Model) o Modelo de Objetos del Documento, permite el acceso y modificación dinámica de los elementos del documento, es decir, contenido, estructura y CSS de una página.
- BOM (Browser Object Model) o Modelo de Objetos del navegador, permite el acceso y modificación dinámica del navegador web.

JavaScript – DOM y BOM



JavaScript – DOM



JavaScript – acceso DOM

Para acceder a los elementos del DOM se puede utilizar el punto . entre el elemento y su padre o sus propiedades, o se puede acceder mediante su id o clase.

JavaScript – acceso DOM

Ejemplo:

```
document.body.style.backgroundColor = 'yellow'
```

Cambia el color de fondo de la etiqueta body mediante una propiedad CSS. Nótese que originalmente la propiedad CSS es background-color pero aquí se transforma a la notación camelCase

JavaScript – acceso DOM

También podemos acceder a un elemento mediante su id, el cual es único en toda la página.

`getElementById('id')`

Ejemplo:

```
<h1 id="titulo">Prueba sitio</h1>
```

```
<input type="button" onclick =  
"document.getElementById('titulo').style.color =  
'red'" value="Enviar">
```


JavaScript – variables

En JS no es necesario indicar el tipo de dato de las variables y para usarlas solo se indica un nombre. En el ejemplo anterior podemos usar una variable.

Ejemplo:

```
<h1 id="titulo">Prueba sitio</h1>  
  
<input type="button" onclick =  
"algo=document.getElementById('titulo');  
algo.style.color = 'red'" value="Enviar">
```

JavaScript – variables

Como se vio en el ejemplo anterior, se puede separar más de una instrucción en JS con un punto y coma:

```
<h1 id="titulo">Prueba sitio</h1>  
<input type="button" onclick =  
"algo=document.getElementById('titulo');  
algo.style.color = 'red'" value="Enviar">
```



JavaScript – Eventos

A continuación, algunos eventos que utilizaremos para ejecutar código JS.

Eventos de teclado:

Evento	Descripción	Aplicable a
onKeyPress	Pulsar una tecla	<body> y elementos de formulario
onKeyDown	Pulsar una tecla y no soltarla	<body> y elementos de formulario
onKeyUp	Soltar una tecla pulsada	<body> y elementos de formulario

JavaScript – Eventos

Eventos del mouse:

Evento	Descripción	Aplicable a
onClick	Pulsar el botón principal y soltar	Todos los elementos
onDbIcClick	Pulsar 2 veces seguidas el botón principal	Todos los elementos
onMouseDown	Pulsar el botón y no soltarlo	Todos los elementos
onMouseUp	Soltar el botón pulsado	Todos los elementos
onMouseMove	Mover el mouse	Todos los elementos
onMouseOver	El mouse entra sobre el elemento	Todos los elementos
onMouseOut	El mouse sale del elemento	Todos los elementos

JavaScript – Eventos

Eventos del body:

Evento	Descripción	Aplicable a
onLoad	Cuando la página se carga completamente	<body>
onUnload	Cuando se abandona la página	<body>
onResize	Al modificar el tamaño de la ventana	<body>

JavaScript – Eventos

Eventos de formulario:

Evento	Descripción	Aplicable a
onFocus	Seleccionar un elemento (recibe el foco)	<button>, <input>, <label>, <select>, <textarea>, <body>
onBlur	Deseleccionar un elemento (pierde el foco)	<button>, <input>, <label>, <select>, <textarea>, <body>
onChange	Deseleccionar un elemento que ha sido modificado	<input>, <select>, <textarea>
onSubmit	Enviar el formulario	<form>
onReset	Limpiar el formulario	<form>
onSelect	Seleccionar un texto	<input>, <textarea>

JavaScript – This

En JS la palabra clave **this** hace referencia a un objeto. El tipo de objeto al que haga referencia, dependerá de donde se la invoque o use, es decir, sería sobre sí mismo.

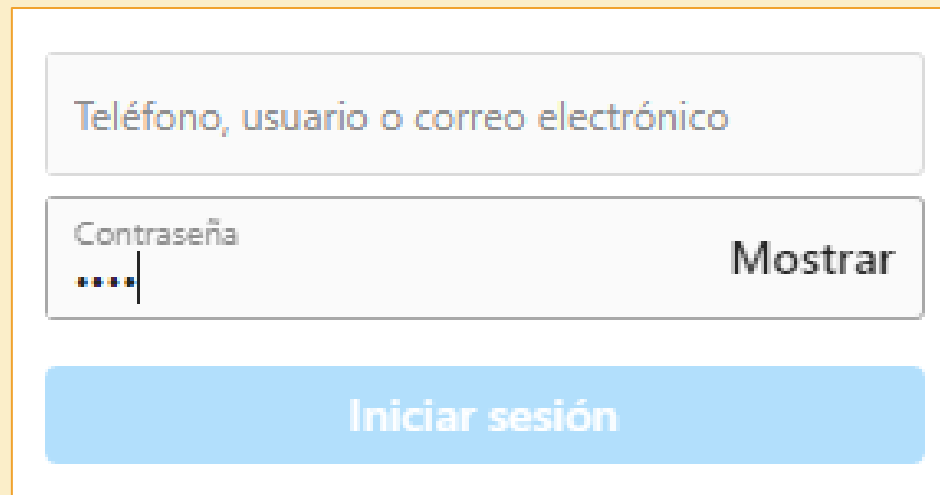
```
<label for="mail">Correo:</label>  
<input type="email" name="correo" id="prueba"  
onkeypress="this.value += ' - '>
```

En este caso al presionar alguna tecla en el campo del mail, le concatenará a este mismo campo, un guion.

This está haciendo referencia al mismo elemento desde donde es invocado.

JavaScript – Ejemplos

En un formulario de inicio de sesión la contraseña que se ingresa no es visible, pero se acostumbra a tener la opción de mostrarla.



A login form UI mockup with a light gray background and rounded corners. It features three main components: a top input field with the placeholder text 'Teléfono, usuario o correo electrónico'; a middle section containing a password input field with the label 'Contraseña' and a 'Mostrar' button to its right, where the password is masked with dots; and a bottom blue button with the text 'Iniciar sesión'.

Teléfono, usuario o correo electrónico

Contraseña

Mostrar

Iniciar sesión

JavaScript – Ejemplos

Podemos incorporar esta funcionalidad con JS

```
<label for="mail">Pass:</label>  
<input type="password" name="pass" id="pass">  
<input type="button" value="Mostrar" onclick="document.  
getElementById('pass').type = 'text'">
```

Pass:

Pass:

Obtenemos el id del input password y cambiamos su tipo de password a text para que sea visible

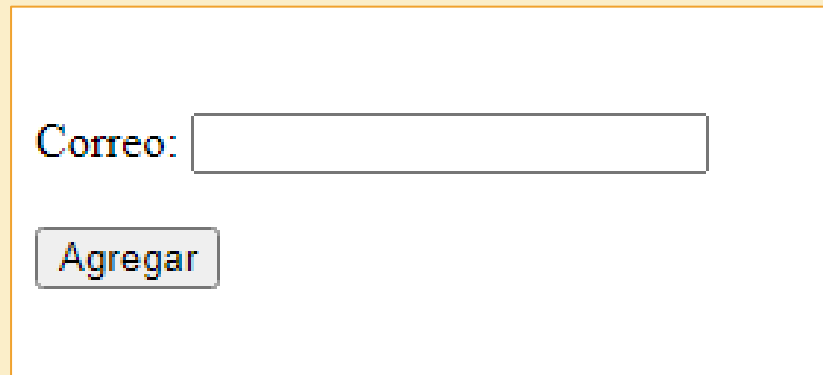
JavaScript – Ejemplos

También se puede hacer el inverso para volver a ocultarlo.

Más aún, no tiene que ser necesariamente un botón, puede ser cualquier otra etiqueta!!

JavaScript – Ejemplos

Podemos cambiar el contenido de alguna etiqueta HTML, por ejemplo, podemos armar una lista de correos con un solo input de tipo email.



Correo:

JavaScript – Ejemplos

Tenemos un section vacío con un id

```
<label for="cor">Correo:</label>
<input type="email" id="cor" name="cor">
<input type="button" value="Agregar" onclick="document.getElementById('lista').innerHTML
+= '<p>'+document.getElementById('cor').value+'</p>'">
<section id="lista"></section>
```

La propiedad innerHTML, agrega el texto entre comillas (que puede ser código HTML) al contenido de la etiqueta con el id lista

Correo:

pepe@unsta.edu

otro@coso.com

JavaScript – Ejemplos

Conversión de tipos - `parseInt`

```
<form action="" method="" name="formula">  
  <label for="pri">1er número:</label>  
  <input type="number" id="pri" name="num1">  
  <label for="sec">2do número:</label>  
  <input type="number" id="sec" name="num2">
```

Tenemos un formulario con 2 input del tipo **number** con la finalidad de tener un botón para sumar los 2 números.

A pesar de ser tipo **number**, su atributo value en realidad lo guarda como **texto**.

JavaScript – Ejemplos

Conversión de tipos - `parseInt`

```
<input type="button" value="Sumar" onclick="document.getElementById('aqui').innerHTML =  
parseInt(document.getElementById('pri').value) + parseInt(document.getElementById('sec').  
value);">  
    </form>  
</section>  
<section id="aqui">  
  
</section>
```

Se debe convertir ese value en un número entero con la función **parseInt**. En este ejemplo, el resultado de la suma se inserta como texto en un sección de id “aquí”

JavaScript – Ejemplos

```
algo = window.prompt("Mensaje","Valor por defecto");
```

Permite el ingreso de datos y lo podemos guardar en una variable.

Esto lo podemos usar dentro de la etiqueta script al comienzo de nuestro código, y luego usar la variable en cualquier lugar o evento.

Ejercicio

Muestre una ventana que solicite el nombre, guarde esa información en una variable.

Realice una página con un botón y un section vacío, al presionar el botón, se debe mostrar el mensaje:

Bienvenido Nombre ← el dato de la variable

El mensaje se debe mostrar dentro del section vacío con formato de título (use h2)